

周子龙

 周子龙 | Modified Today

Email: zzl20001209@gmail.com Phone: 18270568907

Education

西安电子科技大学 - 计算机科学与技术 - 本科 - 2019/09 - 2023/07

Papers

Magnus: A Holistic Approach to Data Management for Large-Scale Machine Learning Workloads | [VLDB 2025](#)

Presentations

A Unified Solution for Data Management and Model Training With Apache Iceberg and Mosaic Streaming | [Databricks Data+AI Summit 2025 @San Francisco](#)

Work Experience

Anuttacon - Software Engineer - Shanghai

2025/08/25 - present

负责 Audio 模态 data pipeline 和 data infra 方面的工作，包括预训练的数据处理流水线、特征提取、存储格式设计和训练 dataloading，后训练相关的数据合成、数据迭标/标注结果管理，覆盖整个预训练和后训练的数据链路。整体数据规模在千万小时级（几十 PB），预训练集群规模在千卡级。

Highlight 1: Audio Data Pipeline 优化

Audio pipeline 早期是一整个 python 脚本，通过落盘 parquets 作为 checkpoint 串起若干 ray dataset 来完成整个数据处理过程，经多轮拆解演进为主链 9 个独立可串可并 Job + 30+ Ray Data 算子库形态（算子按需重组成 Task，Task 串成参数化的可执行 Job），中间结果通过落表 Iceberg 持久化，方便跟踪处理进展、数据探查。

Ray Data Autoscaler 改造：针对 abab 主链中多 GPU/CPU 混合长链路任务（如 ASR、Diarization、DNSMOS、SNR、PANNs 等 6-stage pipeline），解决默认 autoscaler 仅基于当前 op input queue 扩缩容、无法感知下游 backpressure 的问题，避免上游 actor pool 盲目扩容导致 Object Store spill、raylet OOM、actor RAM 累积及 fractional GPU 共卡显存 OOM。通过引入下游 backpressure 感知的扩缩容策略和批量扩容机制，将任务从 **OOM 频发优化到零 OOM**，集群冷启动从 **分钟级缩短到数十秒**。

ResourceAllocator 资源分配框架：针对 30+ 算子 GPU/CPU 配比、单条耗时和内存峰值差异大的问题，抽象算子自描述资源模型，将手写 `(num_gpus, concurrency)` 魔数配置转化为基于 CPU/GPU 预算的三阶段约束求解，自动生成 `(min_parallelism, max_parallelism, num_cpus, num_gpus)` 配置，系统性避免资源 spec 错位。典型 CPU 算子 CalDer 并发从受 GPU 数限制的约 **8 并发/节点**提升到 CPU 数限制的约 **150~200 并发/节点**，新业务接入调参轮次从 **3~5 轮**降为 **0**。

Operator Fusion 定制化：针对 Ray Data 自动 fusion 在异构 `num_gpus / concurrency` 算子间容易失效的问题，识别并治理 wav binary 在 Object Store 中反复跨节点传输、序列化和反序列化造成的内存抖动。在 label quality 长链路中，通过手动 fusion 将列变换、文件加载、模型推理和列清理合并到更少的物理算子中执行，减少中间数据落 Object Store 的次数。最终物理算子数从 **5 个减少到 2~3 个**，Object Store 占用降低 **50%+**，wav binary 往返次数从 **4 次以上减少到 1 次**。

Highlight 2: Pretrain 反馈驱动管线迭代

问题：abab pipeline 初版缺少 dedup 和静音检测能力，算法侧训练监控发现 global train loss 收敛异常，并在 A2A eval 中出现 silence-heavy 生成模式。结合 40-rank DP allreduce 下单 rank 异常会被稀释的特点，判断问题来自 dataset-wide 系统性数据污染，而非局部 batch 异常。进一步联合算法团队通过 Spark SQL 抽样、音频人工复核、S3 codes 分布分析和时间戳 cross-reference，定位出两类主要问题：一是 file-level / part-level 重复样本污染，个别 dataset 重复率峰值达 **11.51%**；二是 outlier 长静音样本扭曲 A2A 训练分布，导致模型 over-fit silence pattern。

Dedup 算子迭代：从 v4 开始引入 **MinHash + LSH + Spark GraphFrame ConnectedComponents** 的 file-level dedup 方案，并基于 **Iceberg-as-state** 维护 dedup 状态，通过 `read_run_ds` 增量机制仅处理新增数据，减少跨 run 重算成本。后续针对 partial overlap 等难以被整文件 Jaccard 捕获的问题，将 dedup 粒度从 file-level 下沉到 part level，提升局部重复识别能力；同时与算法侧抽样验证同文本不同 speaker 的误仿比例，在效果和重算成本之间完成 tradeoff 决策。

静音检测与训练侧过滤：新增 **FSMN-VAD** 静音检测算子，在管线侧输出 `silence_segments_list / silence_ratio / max_continuous_silence` 并写入 segments 状态表，采用「只检测不过滤」策略，将过滤决策下放给训练侧 task-aware processor。A2A 任务使用 sample-level 多档阈值过滤 silence-heavy outlier，并在 token-level 基于 S3 silence codec token 识别连续静音 run，将最长连续静音输出限制到约 **0.72s**；A2T 任务则保留 silence diversity，以贴近生产录音分布。

收益：file-level dedup 后整体真实重复率降至 **2.02%**，global loss 曲线明显变稳；part-level dedup 后整体重复率进一步降至 **<1%**，多数 dataset 降至 **0.0X%**，残留 partial overlap 基本消除；静音检测与过滤上线后，管线侧 sample-level 过滤约 **3%** A2A silence-heavy outlier，训练侧 token-level run cap 纠正 silence prior，缓解 A2A 长 horizon 生成中的 silence-heavy 输出问题，同时保留 A2T 所需的真实语音分布。

Highlight 3: Pretrain Dataloader 优化 + 重构

问题：235B MoE 训练 baseline 存在 **MFU 偏低**、**global loss 偶发 spike**、**整体抖动较大** 等问题。难点在于多重不均衡叠加：

- rank 间样本 seqlen 分布方差大**：token 总量虽均衡，但单 rank 内样本长度差异会导致 attention $O(L^2)$ 计算时间出现 cross-rank 长尾，进而使 DP allreduce 等待长尾 rank，快 rank GPU 空闲。
- rank 间 / step 间 TASK 分布漂移**：同 step 各 DP rank 看到不同 task mix，跨 step 全局 task ratio 也会漂移，导致 loss 不稳定。
- online 数据处理浪费计算**：online tokenize / mel STFT 在被过滤 sample 上产生大量无效 CPU 计算，INTERLEAVE 整体淘汰率 **30%~50%**，dataloader 成为 GPU step 前瓶颈。

解决：4 阶段渐进优化，先在 **online** 范式内改良，再切换到 **offline plan** 架构。

- online 范式内优化**
在 sampler 中加入 **balanced sampling on top of cycling**，突破 AncSampler chunk-based cycling 的 periodicity；引入 **DP deterministic per step**，保证同 step 各 DP rank 拿到一致 task 分布；同时打散 cross-rank cycling alignment。配合 **PartSplitter** 将 Part duration 从 **9~10min** 降到 **2~3min**，降低样本 seqlen 方差，使每个 rank 内 seqlen 分布更紧，从而缓解 attention $O(L^2)$ 计算时间的 cross-rank 长尾。该阶段指标有改善，但 online 范式可调空间有限，因此开始切换架构。
- 切换到 offline plan 范式**
将 **12 层过滤、tokenize、mel STFT、序列拼接** 从训练热路径整体搬到离线一次性完成。
设计并实现 Planner 离线遍历 **18 亿+ 样本**，进行 4 阶段装箱和 accumulated remainder tracking，精确控制 task ratio 与跨 rank token 均衡；训练时直读 mmap + lz4 解压，零额外计算。
自研 **AncStore format**，替代列存 parquet，解决 footer 膨胀和多次 S3 GET 问题；**WindowPrestager** 后台预下载覆盖远端 IO 延迟；**Mel Store** 通过 24 字节 ref 跨 task 复用，并支持 speaker-merge 非连续切片。
- Planner V1 → V2 → V3 迭代**
V1 使用纯 **LPT** 做跨 rank 分配；V2 引入 **Swap 后优化精修**，通过 max/min rank pair local search 将 CV 推到 **0.005%**；V3 进一步处理 audio modality 特有的 mel-heavy sequence attention $O(L^2)$ 计算不均衡问题，因为 token-balanced 本身仍不够，引入 **mel-aware LPT + per-MB mel cap + two-level packing**。
- Byte-equal Verifier 兜底**
AncStore 直接承载训练读取，任何不确定性都会导致训练不可复现、debug 困难。不确定性来源分散在 **fork (Dataloader worker)**、**S3 IO**、**多线程**、**actor pool** 四类。
实现 **Ray-distributed double-run subtract byte** 对比，定位并修复 **fork safety**、**S3 retry race**、**actor pool 顺序敏感** 3 类 bug；配套 **fork-safety contract (reinit_after_fork) + thread-local handle + RWLock pool** 保证 AncStore 确定性，并将 verifier 沉淀为后续改动的 regression test。

收益：

- MFU 从 **6~7 ±4** 提升到稳定 **9~10 ±0.5**。
- loss 从 **-1.4 抖动 spike** 转为平滑下降到 **-1.3**。
- 跨 40 rank token CV 达到 **0.005%**，padding ratio **0.09%**。

ByteDance - Software Engineer - Hangzhou

2023/07/01 - 2025/08/25

负责 **Magnus Lake**（基于 Apache Iceberg 增强的数据湖系统）多个模块的功能开发、日常维护和技术演进，覆盖 Feature Store、大模型训练数据管理、多模态数据存储、训练 Dataloader 等场景。

场景 1：Feature Store

基于湖的 **Branching、Column-Level Update、Parquet 列存** 等能力，替代原有 PB 行存格式，在稀疏大宽表（10k~100k columns）场景下优化读写效率和存储成本。主要支撑搜广推模型训练样本宽表和 LRM（Large Recommendation Model）样本存储，当前存储规模 **5+ EB**，最大单表 **百 PB 级**。

特征大表 Join Label 小表性能优化

- 问题**：在 PB 级特征宽表 Join TB 级 Label 小表的样本生产场景中，双方 bucketing 策略不满足 bucket join 条件，且 Label 表不满足广播条件，导致 Join 需要对大表和小表同时 shuffle，性能开销极高。
- 方案**：利用湖的 **Branching** 和 **Partial Update** 能力，将 Join 转换为 **update to temp branch + filter scan**，消除大表 shuffle。
- 收益**：在该业务场景下 Join 性能提升 **10x**。

Parquet 元数据优化

- 问题**：Parquet `FileMetaData` 空间复杂度为 $O(\text{num_row_groups} * \text{num_columns})$ ，且 footer 基于 thrift 编码，需要一次性完整解析。在宽表训练场景中，单次训练通常只读取几万列中的一千多列，但仍需加载完整 footer，导致严重内存和启动开销，单文件元数据存储最高可达 **GB 级**。

- 方案：重构 **Arrow Parquet 元数据模块**，并为存量数据生成优化后的 metadata index，支持按 **Column** 和 **RowGroup** 按需解析元数据。
- 收益：元数据内存开销降低约 **一个数量级**，启动性能提升 **40x**。

Compaction 性能优化

- 问题：原 Compaction 使用 Java Parquet Writer，将 Arrow Batch 按行写出，对列存内存布局不友好，CPU Cache 利用率低；同时按 batch 读取会产生大量随机 HDFS PRead，影响整体 Compaction 吞吐。
- 方案：实现 **Native Rewriter**，直接将 Arrow Batch 写出到 Parquet 文件；同时引入文件本地化策略，先下载到本地 SSD 后进行随机读，降低 HDFS 随机读开销。
- 收益：多个业务场景下 Compaction 性能平均提升 **4.8x**。

Iceberg 元数据优化

- 问题：Iceberg manifest file 按写入顺序组织 entries，在大宽表分区过滤查询中，需要完整扫描 manifest 才能找到目标分区数据。当数据规模大且每次写入覆盖所有分区时，即使只查询单个分区，Planning 也可能耗时几十分钟。
- 方案：对 manifest files 按分区排序，并建立 **Avro data block 粒度稀疏索引**，在 Planning 阶段跳过无关 data blocks；针对全表扫描场景，通过 Spark 作业分布式解析元数据，突破 Iceberg 单机多核 planning 的限制。
- 收益：多个业务场景下 Planning 耗时降低约 **一个数量级**；全表扫描场景中，大部分 Planning 耗时压缩到 **1 分钟以内**。

场景 2：Data Management for Large Models Training

将大模型训练链路中 Pretrain、RL、RM、Evaluation 等阶段的数据统一管理在湖上，使训练、数据洞察、检索查询等需求复用同一份数据，减少冗余存储。基于湖的元数据体系提供 **数据版本、血缘、权限、审计、治理、数据沉降、跨机房迁移** 等能力，并针对长文本、多模态数据优化训练读取性能。

主要支持文本大模型 Pretrain/RL/RM/Evaluation 数据集、多模态大模型如 **Seed1.5-VL** 预训练数据集的入湖管理，以及与 Seed 训练 Infra 团队合作开发高效 Dataloader。

面向多模态数据的 Parquet 扩展

- 问题：多模态数据（图片、音频、视频等）直接以 Blob values 存入 Parquet 时，会受到 Parquet paging、encoding、compression、RowGroup 机制影响，导致存储效率低、随机访问困难；若 Parquet 中只存 URL，将 Blob values 放在对象存储，又会带来对象存储小文件问题和 QPS 瓶颈。
- 方案：设计并实现 **Blob files + Parquet refs** 的存储方案：将 Blob values 存储到独立 blob files 中，Parquet 中的 blob column 仅保存 `offset + length` 引用；blob files 的 `location`、`file size` 等信息记录在 Parquet footer 中，生命周期与 Parquet file 保持一致。读取时先过滤 refs，再从 blob files 读取实际数据，并在顺序扫描时对 refs 做 **IO coalesce**。
- 收益：消除训练场景中的独立打包环节，训练吞吐与打包方案持平；在评测场景下，相比旧 Parquet 方案，采样读取性能提升接近 **两个数量级**，内存占用降低 **两到三个数量级**；扫描性能提升 **20%~30%**，内存占用降低 **10%~20%**。

训练启动优化

- 问题：在 VLM 和 t2/t2v 训练场景中，source 数量可达数百到上万。每个 data worker 都需要获取所有 sources 的 file paths、row counts 等元信息用于 sharding 和 shuffling。直接使用 pyiceberg planning 会构造大量对象，带来极高内存开销和启动耗时；即使 32 cores 并发解析 manifest，也可能需要几十分钟，严重影响训练 ETTR。
- 方案：针对训练场景设计轻量级 **planning & data loading** 链路：为每个数据集生成 metadata index，记录 files 的 location 和 row counts；训练启动时将 metadata index 从 HDFS 拉取到本地 SSD，workers 直接基于 index 进行 sharding 和 shuffling，并通过 **C++ native reader** 按 `file path + row id` 读取样本，绕过 pyiceberg planning，避免构造海量对象；所有 workers 共享磁盘上的 metadata index。
- 收益：在 VLM 万卡作业中，启动时间降低到 **30s**（从作业初始化完成到打出第一条 loss），整体内存占用降低约 **40%**。

Pipelined Prebuffer

- 问题：Arrow Parquet reader 的 prebuffer 会在文件打开时提交所有待读 ColumnChunks 的 IO ranges，并在整个文件读取完成后才释放内存。在 VLM 场景下，开启 prebuffer 虽然能提升吞吐，但整体内存占用高且不可控。
- 方案：调整 Arrow API 调用方式，将单文件读取拆分为多个 row range；实现 **page 级 prebuffer**，每次仅预取当前 row range 所需 pages。
- 收益：在 VLM 场景下，吞吐与朴素 prebuffer 基本持平，内存占用降低约 **40%**。

Dynamic Prefetcher

- 问题：Dataloader 每个 iteration 会根据用户配置和 tokens 消费情况，从多个 sources 中动态消费数据并组装 batch。由于不同 sources 消费存在倾斜，高频 sources 容易因 prefetch 不足阻塞训练，低频 sources 则可能因消费不及时占用过多内存。
- 方案：设计并实现动态 prefetching 机制，根据不同 sources 的 `get_item` 调用情况动态调整 prefetch buffer size；当连续 N 次读取被阻塞时，自动调大对应 source 的 prefetch buffer。
- 收益：在 VLM 场景下，消除了优化前 Dataloader `get_batch data` 周期性毛刺。

Byte-sized Batches

- 问题：传统 reader 通常基于行数配置 batch size，但不同数据集样本大小分布差异大。在固定 memory quota 下，用户很难为不同数据集设置合适的 batch size，容易出现 batch 过大导致内存超限，或 batch 过小导致吞吐不足。
- 方案：支持基于 **byte size** 配置 batch 大小：通过文件元数据获取 decompressed size 和 row count，估算目标 byte size 对应的行数，使读取出的 batch 大小接近用户设定值。
- 收益：降低用户调参成本，使多数据集训练场景下的数据读取粒度更稳定、内存使用更可控。

ByteDance - Software Engineer Intern - Hangzhou

2023/02/06 - 2023/07/01

负责 Spark Native Engine 的早期调研、设计、开发和上线，以及 Spark 内核功能迭代和问题排查。

Spark 内核功能开发 / 问题排查

- 问题：部分 Spark 作业中，WholeStageCodegen 生成的算子 method size 过大，导致 JIT 失效并引发性能骤降；同时 Spark 未能按预期 fallback 到 pipeline execution。
- 方案：定位并修复 method size 过大导致的 JIT 失效问题，同时排查出 fallback 失效原因是 ClassLoader 不一致导致相关参数未生效，并上线修复版本。
- 收益：若干问题作业平均耗时从 **小时级降低到分钟级**。

Native Engine 函数 / 表达式支持

- 问题：Velox 原生支持的函数和表达式无法完全覆盖站内 Spark SQL / HiveUDF 使用场景，限制 Native Engine 在复杂 ETL 作业中的覆盖率。
- 方案：集成站内 **sonic json** 库，实现 json 类型相关函数和表达式，并支持 json path 解析结果缓存、merge get_json_object 等优化；同时在 Velox 上实现 UDF 框架，支持站内 HiveUDF。
- 收益：线上数十个 ETL 高优任务平均加速 **150%~200%**，个别任务加速 **400%+**。

Native Prefetch

- 问题：原 Spark prefetch 在 task 级别实现，每个 task 构造独立线程池；当 task 只有少量文件时容易退化为顺序读取，且 prefetch 数据常驻内存，带来额外资源和内存开销。
- 方案：在 Native Parquet Reader 中实现 **RowGroup 粒度 prefetch**，提升文件数较少场景下的并行读取能力；将线程池从 task 级改为 executor 级，由容器内所有 tasks 共享；同时支持基于 memory quota 的 spill 和 eviction。
- 收益：提升 Native Parquet Reader 读取并行度，降低线程池和 prefetch buffer 带来的资源开销，使内存占用更可控。

MatrixOrigin - Software Engineer Intern - Shanghai - 2021/07-2022/05

负责开源 OLAP/HTAP 存储引擎的设计和实现：<https://github.com/matrixorigin/matrixone/commits?author=zz1200012> repo1 repo2 repo3